

# FXR60 - Impinj TagProtect: How To Set It Up

**Working procedure for Impinj TagProtect (Protected Mode) over the FXR60 REST API**, including the 32-bit access-password step the current documentation requires but does not explain.

|           |                                                           |
|-----------|-----------------------------------------------------------|
|           |                                                           |
| Reader    | FXR60, reader app 5.0.7                                   |
| Endpoints | PUT /cloud/mode, PUT /cloud/impinjGen2X, PUT /cloud/start |
| Tested    | 2026-09-06, live reader, real tags                        |
| Status    | [OK] Confirmed working on Impinj Monza 4i - see §7        |

## 1. Overview

TagProtect makes a tag RF-silent - it stops answering Gen2 commands unless the reader supplies the correct 32-bit password.

Four steps, in this order:

- |                              |                                            |
|------------------------------|--------------------------------------------|
| A. Identify the chip         | <- TagProtect is Impinj-only               |
| B. Assign + write a password | <- the tag needs one BEFORE step C         |
| C. Verify the password       | <- do not skip; step D succeeds regardless |
| D. Configure + apply         | <- two calls, not one                      |

Three things are easy to get wrong and are the reason this document exists:

1. **The tag needs a 32-bit access password written to it first.** The docs state this as a prerequisite but never say how to choose one, where it goes, or how to write it. See §4 - and note a **wrong password fails silently with HTTP 200**.
2. **`PUT /cloud/impinjGen2X` only saves the config.** You must then start an inventory with `applyImpinjGen2X: true` to activate it.
3. **The chip must be Gen2X-capable.** Not every Impinj part supports Protected Mode, and a non-Gen2X tag produces no effect and no error - which reads exactly like a broken API. See §7.

## 2. Prerequisites

| What                                                | Why                                                                               |
|-----------------------------------------------------|-----------------------------------------------------------------------------------|
| An Impinj tag whose silicon supports Protected Mode | TagProtect is Impinj-proprietary - verify per §3                                  |
| Tag data reaching an endpoint you can read          | Access results arrive in the tag stream, not in the HTTP response                 |
| The tag inside reliable read range                  | Every memory operation runs during an inventory                                   |
| A bearer token                                      | GET /cloud/localRestLogin - tokens expire quickly, mint one per call in long runs |

### Routing tag data somewhere readable

If the reader publishes to AWS IoT (or anywhere you cannot read), point the data endpoint at a local MQTT broker for the duration:

```
PUT /cloud/config
{"READER-GATEWAY":{"endpointConfig":{"data":{"event":{"connections":[{"name":"DATA_MQTT","type":"mqtt",
"options":{"enableSecurity":false,
"endpoint":{"hostName":"<your-broker>","port":1883,"protocol":"tcp"},
"additional":{"cleanSession":true,"clientId":"fxr60-data",
"keepAlive":60,"qos":0},
"publishTopic":["fxr60/tevents"],"subscribeTopic":[]}]}}}}}}
```

Then watch it:

```
mosquitto_sub -h <your-broker> -p 1883 -t 'fxr60/tevents'
```

Capture your baseline first, so you can restore afterwards:

```
GET /cloud/mode
GET /cloud/impinjGen2X
GET /cloud/config
```

### 3. Step A - Identify the chip

TagProtect is not universal across Impinj silicon, and an **E280** EPC prefix tells you nothing about the vendor. Read the TID:

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[1],"transmitPower":30,
"query":{"sel":"ALL","session":"S1","tagPopulation":256,"target":"A"},
"accesses":[{"type":"READ","config":{"membank":"TID","wordPointer":0,"wordCount":6}}],
"tagMetadata":["ANTENNA","RSSI"]}
```

```
PUT /cloud/start {}
... wait ~12 s ...
PUT /cloud/stop
```

The TID arrives in **accessResults**. Decode it:

|                          |      |                                    |
|--------------------------|------|------------------------------------|
| byte 0                   | E2   | ISO/IEC 15963 class                |
| bytes 1-2, upper 12 bits | MDID | mask designer ID -> 0x801 = Impinj |
| bytes 2-3, lower 12 bits | TMN  | tag model number                   |

Example - **e28011c12000112dbc930317**:

| Field | Value | Meaning    |
|-------|-------|------------|
| MDID  | 0x801 | Impinj     |
| TMN   | 0x1C1 | Monza X-2K |

| MDID          | Verdict                                 |
|---------------|-----------------------------------------|
| 0x801         | Impinj - continue                       |
| anything else | not Impinj - TagProtect will never work |

**Then check the part against Impinj's Gen2X tag specification** (<<http://www.impinj.com/Gen2X>>). Decoding the TID tells you *which* chip you have; it does **not** tell you whether that chip implements Protected Mode. Zebra's MAUI Gen2X tutorial makes this a prerequisite.

In our field, of 17 tags with **E280** EPCs, the strongest turned out to be MDID **0x806** - a different vendor entirely. Decode

before you choose.

## 4. Step B - Assign and write the access password

**Check first - your tags may already have one.** Both tags in our confirmed run carried a factory password (**12348765**). Read RESERVED words 0-3 (step C) before writing anything; if a password is already set, use it and skip the write entirely.

### 4.1 Choosing a password

The access password is a **32-bit value written as exactly 8 hexadecimal characters**. What the API enforces, tested:

| Rule                 | Enforced?        | Error if broken                                         |
|----------------------|------------------|---------------------------------------------------------|
| Exactly 8 characters | [OK] yes         | tagProtect.password must be 8 hex characters (32-bit)   |
| Hex only (0-9, A-F)  | [OK] yes         | tagProtect.password contains invalid hex character: 'Z' |
| Case                 | [-] ignored      | abcdef12 and ABCDEF12 both accepted                     |
| Not all zeros        | [X] not enforced | 00000000 is accepted - see the warning below            |

So **A1B2C3D4**, **12348765** and **deadbeef** are all valid inputs. The range is **00000001-FFFFFFFF**, i.e. ~4.29 billion values.

[!] **Never use `00000000`.** On a Gen2 tag a zero access password means *"no password set"* - the tag treats access as unlocked. The API accepts `{"password": "00000000"}` with HTTP 200 and no warning, but it cannot protect anything. This is a real trap: the call looks successful.

#### Guidance for choosing:

| Do                                                   | Don't                                                                            |
|------------------------------------------------------|----------------------------------------------------------------------------------|
| Use a random 8-hex value per deployment              | Don't use 00000000, FFFFFFFF, 12345678 or other guessable patterns               |
| Record it somewhere you will still have in a year    | Don't rely on reading it back later - a properly locked tag will not disclose it |
| Consider one password per site or batch, not per tag | Don't give every tag a unique password unless you have a system to track them    |

Generate one:

```
openssl rand -hex 4 | tr 'a-f' 'A-F' # e.g. 7F3A91C2
```

**One password per batch is usually the right trade-off.** A single password for all tags is simple but is a single point of compromise; a unique password per tag is safest but means you must map EPC -> password and never lose it. Per-site or per-batch keeps the mapping small enough to manage.

### 4.2 A wrong password fails silently

This is the most important thing to know about password handling, and it is tested:

The tag holds **12348765**. We deliberately sent **DEADBEEF**:

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"enableTagProtection",
  "password":"DEADBEEF",
  "tagID":"e28011b0a5050076c4d7530a"}}
```

```
-> 200 Success: Gen2X configured. Use applyImpinjGen2X flag in start command to apply features.
```

Then applied with **applyImpinjGen2X: true** and inventoried three times:

**HTTP 200, no error anywhere, and nothing happened.** The reader has no way to tell you the password did not match the tag, because TagProtect returns no per-tag result.

Consequences for your process:

1. **Always read RESERVED words 2-3 first** and use the value that is actually on the tag (step C).
2. **Always verify by observation** - inventory the tag after applying and confirm it disappeared. A 200 proves nothing.
3. If protection appears not to work, **suspect the password before suspecting the firmware.**

### 4.3 Where the password lives, and writing it

The Gen2 RESERVED bank layout is fixed by the EPC Gen2 specification:

| Words | Contents                              |
|-------|---------------------------------------|
| 0-1   | Kill password (32 bits)               |
| 2-3   | Access password (32 bits) <- this one |

So write 8 hex characters (2 words = 32 bits) at **wordPointer: 2**:

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[1],"transmitPower":30,
 "filter":{"value":"333311112222333344445555","match":"prefix","operation":"include"},
 "accesses":[{"type":"WRITE",
               "config":{"membank":"RESERVED","wordPointer":2,"data":"A1B2C3D4"}}]},
 "tagMetaData":["ANTENNA","RSSI"]}
```

```
PUT /cloud/start {}
... wait ~12 s ...
PUT /cloud/stop
```

The write executes **when the tag is singulated**, not when the PUT returns. Expected **accessResults** in the tag stream:

```
x76 ["SUCCESS"]
x4  ["Error after 0 successful words written: CRC error in tag response"]
x2  ["Error after 0 successful words written: Tag did not respond"]
```

Retries are normal RF behaviour - the **SUCCESS** count is what matters.

[!] **The `filter` is not optional in practice.** Without it the write applies to **every tag in the field**. There were 24 tags present during our testing; all of them would have been given this password. Use the full EPC as a **prefix** match to target exactly one tag.

### Three payload mistakes to avoid

| Sent                           | Response                               |
|--------------------------------|----------------------------------------|
| "match":"equals"               | 422 - only prefix, suffix, regex exist |
| query and filter with prefix   | 422 - they cannot be combined          |
| "wordCounter" in a READ config | 422 - the field is wordCount           |

The last one matters because the spec's own **with\_accesses** example uses **wordCounter**, which the firmware rejects.

### 4.4 Changing a password later

Writing to RESERVED words 2-3 again overwrites the old value - there is no "change password" operation, it is just another write:

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[1],"transmitPower":30,
"filter":{"value":"<EPC>","match":"prefix","operation":"include"},
"accesses":[{"type":"WRITE",
"config":{"membank":"RESERVED","wordPointer":2,"data":"<NEW-8-HEX>"}},
"tagMetadata":["ANTENNA","RSSI"]}
```

[!] **Unprotect the tag first.** Change the password on a tag that is currently protected and you may be left holding a password the tag no longer accepts, with no way to unprotect it. Sequence: **disableTagProtection** -> apply -> write the new password -> verify -> re-protect.

Note the REST **accesses** WRITE config has **no `AccessPassword` field** (the MAUI SDK does have one). On a tag whose memory is already access-locked, REST may therefore be unable to authenticate the write at all - untested, and listed as gap 5 in §10.

## 4.5 If you lose the password

There is no recovery path through this API.

| Situation                        | Recoverable?                                                                        |
|----------------------------------|-------------------------------------------------------------------------------------|
| Password lost, tag not protected | [OK] Read RESERVED w2-3 and recover the value - as long as the bank is not locked   |
| Password lost, tag is protected  | [X] The tag is RF-silent. No reader can address it, so nothing can be read or reset |

That second row is why the password must be recorded outside the tag before you protect anything. A protected tag with a lost password is effectively scrap.

## 5. Step C - Verify the password landed

**Do not skip this.** Every later **tagProtect** call returns HTTP 200 whether the password is right or wrong, so this read is the only checkpoint you get.

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[1],"transmitPower":30,
"filter":{"value":"333311112222333344445555","match":"prefix","operation":"include"},
"accesses":[{"type":"READ","config":{"membank":"RESERVED","wordPointer":0,"wordCount":4}}],
"tagMetadata":["ANTENNA","RSSI"]}
```

Run an inventory, then read **accessResults** - 16 hex characters covering words 0-3:

|        | accessResults    | Kill (w0-1) | Access (w2-3) |
|--------|------------------|-------------|---------------|
| before | 0000000000000000 | 00000000    | 00000000      |
| after  | a1b2c3d4a1b2c3d4 | a1b2c3d4    | a1b2c3d4 [OK] |

**Chip-specific note.** On the Monza X-2K, writing 2 words at **wordPointer: 2** also changed words 0-1 (the kill password). The Monza R6 and a non-Impinj tag behaved normally, changing only the access password. Setting a *kill* password by accident is worse than an access password, so read words 0-3 back and check both when working with unfamiliar stock.

## 6. Step D - Configure and apply

## D1. Configure

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"enableTagProtection",
               "password":"A1B2C3D4",
               "tagID":"333311112222333344445555",
               "enableShortRange":false}}
```

```
-> 200 {"message":"Success: Gen2X configured. Use applyImpinjGen2X flag in start command to apply features."}
```

**This only saves the configuration.** Confirmed immediately after the 200:

```
GET /cloud/impinjGen2X -> the tagProtect object, stored [OK]
GET /cloud/status      -> "impinjGen2X":{"feature":"none","isActive":false} NOT active
```

## D2. Clear the mode filter

Your mode still carries the single-tag **filter** and the memory **accesses** from steps B and C. Leave them in place and the inventory sees only that one tag. Reset to a plain inventory:

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[1],"transmitPower":30,"tagMetaData":["ANTENNA","RSSI"]}
```

## D3. Apply

```
PUT /cloud/start
{"applyImpinjGen2X":true}
```

```
-> 200 {"message":"Success: Radio started with Gen2X. Note: Gen2X will not auto-start on reboot (normal mode will resume)."} 
```

Now it is live:

```
GET /cloud/status -> "impinjGen2X":{"feature":"tagProtect","isActive":true} [OK]
```

[!] Send ``applyImpinjGen2X: true`` on every start. The activation is per-inventory-session. After `PUT /cloud/stop` it reverts to `{"feature":"none","isActive":false}`, and a later plain `start {}` will **not** re-apply it even though the config is still saved.

## The four actions

The schema is a **oneOf**, and ``tagID`` placement is the difference:

| Action               | Scope      | tagID           |
|----------------------|------------|-----------------|
| enableTagProtection  | one tag    | required        |
| disableTagProtection | one tag    | required        |
| enableTagVisibility  | the reader | must be omitted |
| disableTagVisibility | the reader | must be omitted |

The **visibility** actions control whether *this reader* can see protected tags. They do not protect or unprotect anything - easy to conflate given the names.

Both rules are enforced, with clear errors:

```
tagProtect.tagID is required for action: enableTagProtection
tagProtect.tagID is not allowed for action: enableTagVisibility
```

`password` is pattern-checked `^[0-9A-Fa-f]{8}$` - exactly 8 hex characters. The validation is genuinely good; `"ZZZZZ  
ZZZ"` returns `tagProtect.password contains invalid hex character: 'Z'`.

## Only one Gen2X feature at a time

```
{"code":3,"message":"... Only one Impinj Gen2X feature can be configured at a  
time (fastID, tagProtect, tagFocus, or tagQuieting)"}
```

You cannot combine `tagProtect` with `fastID`, `tagFocus` or `tagQuieting` in one request.

## 7. Confirmed working

**TagProtect works.** Verified on 2026-09-07 against two Impinj **Monza 4i** tags (MDID `0x801`, TMN `0x1B0`) on an FXR90, reader app 5.0.4.

Five 10-second inventories per phase, with an identical control tag never named in any request:

| Phase                      | Gen2X applied | Target seen | Control seen |
|----------------------------|---------------|-------------|--------------|
| Baseline                   | none          | 5/5         | 5/5          |
| After enableTagProtection  | tagProtect    | 1/5         | 5/5          |
| Gen2X cleared entirely     | none          | 0/5         | 5/5          |
| After enableTagVisibility  | tagProtect    | 5/5         | 5/5          |
| After disableTagProtection | none          | 5/5         | 5/5          |

The third row is the decisive one: with **no Gen2X configuration loaded at all**, the target stays invisible. The protection is written into the tag's silicon, so the tag is genuinely RF-silent to any reader that does not supply the password - exactly as documented. Row 4 then shows an authorised reader seeing it again.

The control tag stayed visible 5/5 in every phase, which isolates the effect to the targeted EPC.

**The chip is what matters.** An earlier run of this same procedure produced no effect at all, because the tags in that field were a non-Impinj part, a Monza R6 (no Protected Mode), and an unverified Monza X-2K. Nothing was wrong with the API - the tags were not Gen2X-capable. **Do step A properly; it is the difference between a working feature and an apparent bug.**

Both test tags already carried a factory access password (`12348765`), so no write was needed - step B was skipped and the existing password used. That is the cleanest path if your tags are already programmed: read RESERVED words 2-3 first and use what is there.

Full measurement, payloads and raw captures: `tagprotect-CONFIRMED-monza4i/TAGPROTECT_CONFIRMED.md`

## 8. Reversing it - order matters

### 8.1 Unprotect first

```
PUT /cloud/impinjGen2X
{"tagProtect":{"action":"disableTagProtection",
               "password":"A1B2C3D4",
               "tagID":"333311112222333344445555"}}
```

```
PUT /cloud/start {"applyImpinjGen2X":true}
... wait ~8 s ...
PUT /cloud/stop
```

## 8.2 Then clear the password

```
PUT /cloud/mode
{"type":"CUSTOM","antennas":[1],"transmitPower":30,
 "filter":{"value":"333311112222333344445555","match":"prefix","operation":"include"},
 "accesses":[{"type":"WRITE",
               "config":{"membank":"RESERVED","wordPointer":0,"data":"0000000000000000"}]},
 "tagMetadata":["ANTENNA","RSSI"]}
```

**wordPointer: 0** with 16 hex characters clears **both** passwords - needed on the X-2K, where the earlier write had also set the kill password. On other chips **wordPointer: 2** with "00000000" is enough.

[!] **Never clear the password before unprotecting.** If you zero the access password on a genuinely protected tag, you can no longer supply the password needed to unprotect it, and the tag stays RF-silent to any reader that does not know the old value.

## 8.3 Restore the reader

```
PUT /cloud/impinjGen2X {"fastID":{"enabled":false}} clear the Gen2X config
PUT /cloud/mode <your baseline mode object>
PUT /cloud/config <your baseline data endpoint>
```

Verify with **GET /cloud/status -> "impinjGen2X":{"feature":"none","isActive":false}.**

## 9. Quick reference

|         |                                                  |                                        |
|---------|--------------------------------------------------|----------------------------------------|
| 1. GET  | /cloud/mode, /cloud/impinjGen2X, /cloud/config   | capture baseline                       |
| 2. PUT  | /cloud/config                                    | route data somewhere readable          |
| 3. PUT  | /cloud/mode READ TID                             | identify the chip                      |
| 4.      | (check the part against Impinj's Gen2X spec)     |                                        |
| 5. PUT  | /cloud/mode READ RESERVED w0, count 4            | check current password                 |
| 6. PUT  | /cloud/mode WRITE RESERVED w2, 8 hex             | assign+write password (never 00000000) |
| 7. PUT  | /cloud/mode READ RESERVED w0, count 4            | VERIFY it landed                       |
| 8. PUT  | /cloud/impinjGen2X tagProtect + password + tagID | configure                              |
| 9. PUT  | /cloud/mode plain inventory                      | clear the filter                       |
| 10. PUT | /cloud/start {"applyImpinjGen2X": true}          | apply                                  |
| 11.     | ... observe ... ideally from a second reader     |                                        |
| 12. PUT | /cloud/impinjGen2X disableTagProtection -> apply | unprotect                              |
| 13. PUT | /cloud/mode WRITE RESERVED w0, zeros             | clear the password                     |
| 14. PUT | /cloud/mode, /cloud/config                       | restore baseline                       |

Steps 5-7 all need **start** -> **wait** -> **stop** around them; memory operations run during singulation.

## 10. Documentation gaps found

Worth raising with Zebra:

| # | Gap |
|---|-----|
|---|-----|



|   |                                                                                                                                                                                                                                                                                                         |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | The access-password prerequisite is stated but never explained. No guidance on choosing a password, where it lives (RESERVED words 2-3), that it is written via a set_mode WRITE access, that it executes during an inventory, or that a filter is required to avoid writing to every tag in the field. |
| 2 | Which Impinj chips support Protected Mode (highest priority). "Use an Impinj tag" is not sufficient. Confirmed working on Monza 4i; produced no effect on a Monza R6. A supported-parts list would have saved a full day of testing.                                                                    |
| 3 | applyImpinjGen2X is per-session. The note mentions reboot behaviour only; in practice the flag must be re-sent on every start.                                                                                                                                                                          |
| 4 | TagProtect results are not observable. set_mode accesses return accessResults per tag; TagProtect returns nothing and logs no outcome. On a non-Gen2X tag this makes "unsupported chip" indistinguishable from "feature broken" - which is exactly the confusion we hit.                                |
| 5 | REST has no AccessPassword on WRITE operations. The MAUI SDK passes one to authenticate a write; the REST accesses WRITE config accepts only membank, wordPointer, data, blockSize. This may mean REST cannot authenticate accesses to a tag that is protected.                                         |
| 6 | tagID placement. Required for the protection actions, forbidden for the visibility actions - worth an explicit callout.                                                                                                                                                                                 |
| 7 | 00000000 is accepted as a tagProtect password. A zero access password means "unset" on a Gen2 tag, so it cannot protect anything - yet the API returns HTTP 200 with no warning. Please reject it.                                                                                                      |
| 8 | A wrong password fails silently. enableTagProtection with a password that does not match the tag returns HTTP 200 and leaves the tag unprotected, with no error and no per-tag result. Integrators cannot tell success from failure without inventorying the tag themselves.                            |
| 9 | filter.match has no equals. Single-tag targeting via a full-EPC prefix works but reads as a workaround. The error message is also malformed: "must be a must be prefix, suffix, or regex".                                                                                                              |

*Procedure written on FXR60 app 5.0.7 (2026-09-06); confirmed working on FXR90 app 5.0.4 with Impinj Monza 4i tags (2026-09-07). Every payload in this document was sent to a live reader; the request/response captures are in the numbered folders alongside this file.*